

HCM360 HRIS

# APIs and Logic

*Request routing, business rules, and inter-module communication*

Version 1.0.0 · April 2026

## 1. Overview

HCM360 exposes its functionality through HTTP routes defined as Flask blueprints. Each functional module is an independent blueprint with its own URL prefix, templates, and service layer. Routes are protected by the access matrix (page + feature + modification tiers) and data is filtered through the privacy service.

The system is primarily server-rendered. A small set of AJAX endpoints return JSON for live editing patterns (access matrix toggles, privacy rules, inline edits). All mutations require a valid CSRF-protected session.

## 2. Blueprint Map

| Blueprint  | URL Prefix          | Module                           |
|------------|---------------------|----------------------------------|
| dashboard  | /                   | Landing dashboard with KPIs      |
| employees  | /employees          | Core HR + 201 file management    |
| attendance | /attendance         | DTR, shifts, overtime            |
| leave      | /leave              | Leave requests + balances        |
| payroll    | /payroll            | Runs, payslips, loans            |
| ess        | /me                 | Employee self-service            |
| mss        | /team               | Manager self-service             |
| orgchart   | /orgchart           | Org hierarchy visualization      |
| wfp        | /workforce-planning | Scenario modeling                |
| dms        | /dms                | 201 documents, certificate queue |
| rsp        | /recruitment        | Requisitions & candidates        |
| pm         | /performance        | Reviews & evaluations            |
| ld         | /learning           | Trainings & certifications       |
| rr         | /rewards            | Awards & praise                  |
| discipline | /discipline         | Cases & sanctions                |
| health     | /health             | Incidents & safety reports       |
| analytics  | /reports            | Dashboards & report builder      |
| ai         | /ai                 | ARIA AI assistant                |
| admin      | /admin              | System configuration             |
| workflow   | /workflow           | Task inbox & approvals           |

## 3. Request Routing Pattern

Every blueprint follows the same layering convention. The routes file handles HTTP concerns (form parsing, redirects, flashes); the service module handles DB access and business logic.

### 3.1 Example: Employee Detail Route

```
@employees_bp.route("/<int:employee_id>")
@login_required
def employee_detail(employee_id):
    profile = svc.get_employee_profile(employee_id)
    if not profile:
        abort(404)

    role_code = session.get("role_code", "")
    is_own = (session.get("employee_id") == employee_id)

    # Field-level privacy masking
    profile["employee"] = apply_privacy(profile["employee"],
                                         role_code, "employee", is_own_record=is_own)
    profile["gov_ids"] = apply_privacy(profile.get("gov_ids", []),
                                         role_code, "government_id", is_own_record=is_own)
    profile["banks"] = apply_privacy(profile.get("banks", []),
                                       role_code, "bank_account", is_own_record=is_own)
    profile["dependents"] = apply_privacy(profile.get("dependents", []),
                                           role_code, "dependent", is_own_record=is_own)

    return render_template("employees/detail.html", profile=profile)
```

## 4. Service Layer Conventions

Service modules accept primitives (IDs, strings) and return plain dicts or lists-of-dicts. They use the shared `get_cursor()` context manager which yields a `psycpg2 DictCursor`, ensures transaction commit/rollback, and returns the connection to the pool.

### 4.1 Database Access Pattern

```
from services.db import get_cursor

def get_employee_profile(employee_id):
    with get_cursor() as cur:
        cur.execute("""
            SELECT e.*, d.name AS department_name, p.title AS position_title,
                   s.full_name AS supervisor_name
            FROM core.v_employees_full e
            LEFT JOIN core.departments d ON d.id = e.department_id
            LEFT JOIN core.positions p   ON p.id = e.position_id
            LEFT JOIN core.employees s   ON s.id = e.immediate_supervisor_id
            WHERE e.id = %s
            """, (employee_id,))
        emp = cur.fetchone()
        if not emp:
            return None
        # ... fetch addresses, gov_ids, banks, dependents
        return {"employee": emp, "addresses": addrs, ...}
```

## 5. Authentication & Session

Login is currently a user-selector dropdown (POC mode). The POST /login/select endpoint sets session["user\_id"]. A before\_request hook re-derives role\_code and employee\_id from the DB on every request, so role changes take effect immediately.

### 5.1 Key Session Variables

| Key         | Type     | Description                                               |
|-------------|----------|-----------------------------------------------------------|
| user_id     | int      | Primary identifier (maps to core.users.id)                |
| role_code   | str      | Derived on each request from core.user_roles + core.roles |
| employee_id | int None | core.employees.id if user is linked to an employee        |
| _csrf_token | str      | Random token regenerated per session                      |

### 5.2 SSO Token Support

A JWT token can be passed as ?sso\_token= query parameter. The token is decoded without signature verification (POC mode), and the username claim is matched against email local-part or display\_name to resolve user\_id.

## 6. Access Matrix Logic

Three-tier RBAC enforced in services/access\_service.py:

### 6.1 Page Access

can\_access\_page(role\_code, path, user\_id=None) looks up the path in core.page\_registry using LIKE matching (so /employees/ matches /employees/123). User-level overrides take precedence; SUPER\_ADMIN always passes.

### 6.2 Feature/Action Access

can\_access\_feature(role\_code, feature\_code, user\_id=None) checks fine-grained actions like PAY\_RUN\_APPROVE, LV\_BALANCE\_ADJUST, EMP\_PRIVACY\_ADMIN. Used for conditional button rendering in templates and final checks in POST handlers.

### 6.3 Modification Access

can\_modify(module\_code, role\_code, user\_id=None) controls whether a role can mutate data in a module. Separate from page access so an EXECUTIVE can view Payroll but not change anything. Configured at /admin/modification-access.

## 7. Privacy Service

`privacy_service.apply_privacy(data, role_code, section, is_own_record=False)` masks sensitive fields per `core.field_privacy_rules`. Masking states:

- **VISIBLE** — pass-through unchanged
- **MASKED** — replaces value with `mask_char` (default `****`)
- **HIDDEN** — removes the key from the dict entirely
- `is_own_record=True` + **EMPLOYEE**: **MASKED**→**VISIBLE**, **HIDDEN** stays **HIDDEN** (own contact info visible, own salary never)

**HR\_ADMIN** and **SUPER\_ADMIN** bypass all masking by default.

## 8. Workflow Engine

Cross-module workflows (leave approvals, OT approvals, discipline cases) use a lightweight engine:

- `workflow_definitions` — named workflows with steps and approver rules
- `workflow_instances` — one row per transaction going through a workflow
- `workflow_steps` — current + history of steps per instance
- `inbox_items` — generated when an approver action is needed

The workflow blueprint at `/workflow/*` lets approvers act on their inbox without leaving the site.

## 9. Reporting & PDF Generation

`services/reporting_service.py` produces PDFs using ReportLab. It consumes views (e.g., `payroll.v_payslip`) and renders standardized templates. The payslip PDF is generated on demand by `GET /payroll/payslips/<id>/pdf`.

## 10. AJAX Endpoints (JSON)

| Endpoint                                      | Method | Purpose                                               |
|-----------------------------------------------|--------|-------------------------------------------------------|
| <code>/admin/access-matrix/toggle</code>      | POST   | Toggle a single page/feature grant for a role or user |
| <code>/admin/privacy/save</code>              | POST   | Upsert a field privacy rule                           |
| <code>/admin/modification-access/grant</code> | POST   | Grant/revoke module-level write access                |
| <code>/api/search?q=</code>                   | GET    | Global search across employees, documents, tasks      |
| <code>/ai/ask</code>                          | POST   | Submit question to ARIA AI assistant                  |
| <code>/orgchart/api/tree</code>               | GET    | Org tree JSON for dynamic rendering                   |

## 11. Error Handling

- 400 Bad Request — form validation failures (re-render form with flash)
- 403 Forbidden — access denied by page, feature, or mod permission
- 404 Not Found — requested resource does not exist
- 500 Internal Server Error — rendered via error template with a request ID for log correlation

All non-200 responses write to `audit_logs.access_log` for forensic review.

## 12. Business Logic Highlights

### 12.1 Philippine Statutory Leave Eligibility

Leave types seeded with `legal_basis` (e.g., RA 11210 Maternity, RA 8187 Paternity, RA 8972 Solo Parent, RA 9262 VAWC, RA 9710 Magna Carta, RA 6727 SIL). Rules encoded in leave service: `gender_restriction`, `max_days_per_filing`, `requires_document`.

### 12.2 Statutory Deductions (SSS/PhilHealth/HDMF/BIR)

Payroll service loads active schedules from ref tables and applies them per bracket. Tables updatable without code changes through `/admin/reference`.

### 12.3 Holiday Premium Pay

attendance service classifies dates against `lv_holidays`; payroll applies multipliers per `h_type` (REGULAR 200%, SPECIAL\_NON\_WORKING 130%, etc.) from `attendance.att_holiday_types`.